

Análisis de Algoritmos: Complejidad

1. Introducción

La resolución práctica de un problema exige por una parte un algoritmo o método de resolución y por otra un programa o codificación de aquel en un ordenador real. Ambos componentes tienen su importancia; pero la del algoritmo es absolutamente esencial, mientras que la codificación puede muchas veces pasar a nivel de anécdota.

A efectos prácticos o ingenieriles, nos deben preocupar los recursos físicos necesarios para que un programa se ejecute. Aunque puede haber muchos parámetros, los más usuales son el tiempo de ejecución y la cantidad de memoria (espacio). Ocurre con frecuencia que ambos parámetros están fijados por otras razones y se plantea la pregunta inversa: ¿cuál es el tamaño del mayor problema que puedo resolver en T segundos y/o con M bytes de memoria? En lo que sigue nos centraremos casi siempre en el parámetro tiempo de ejecución, si bien las ideas desarrolladas son fácilmente aplicables a otro tipo de recursos.

Para cada problema determinaremos una medida N de su tamaño (por número de datos) e intentaremos hallar respuestas en función de dicho N. El concepto exacto que mide N depende de la naturaleza del problema. Así, para un vector se suele utilizar como N su longitud; para una matriz, el número de elementos que la componen; para un grafo, puede ser el número de nodos (a veces es más importante considerar el número de arcos, dependiendo del tipo de problema a resolver); en un archivo se suele usar el número de registros, etc. Es imposible dar una regla general, pues cada problema tiene su propia lógica de coste.

2. Tiempo de Ejecución

Una medida que suele ser útil conocer es el tiempo de ejecución de un programa en función de N, lo que denominaremos $T(N)$. Esta función se puede medir físicamente (ejecutando el programa, reloj en mano), o calcularse sobre el código contando instrucciones a ejecutar y multiplicando por el tiempo requerido por cada instrucción. Así, un trozo sencillo de programa como

```
S1;  
for (int i= 0; i < N; i++)  
    S2;
```

requiere

$$T(N) = t_1 + t_2 * N$$

siendo t_1 el tiempo que lleve ejecutar la serie "S1" de sentencias, y t_2 el que lleve la serie "S2".

Prácticamente todos los programas reales incluyen alguna sentencia condicional, haciendo que las sentencias efectivamente ejecutadas dependan de los datos concretos que se le presenten. Esto hace que más que un valor $T(N)$ debamos hablar de un rango de valores

$$T_{\min}(N) \leq T(N) \leq T_{\max}(N)$$

los extremos son habitualmente conocidos como "caso peor" y "caso mejor". Entre ambos se hallará algún "caso promedio" o más frecuente.

Cualquier fórmula $T(N)$ incluye referencias al parámetro N y a una serie de constantes " T_i " que dependen de factores externos al algoritmo como pueden ser la calidad del código generado por el compilador y la velocidad de ejecución de instrucciones de la computadora que lo ejecuta. Dado que es fácil cambiar de compilador y que la potencia de los ordenadores crece a un ritmo vertiginoso, intentaremos analizar los algoritmos con algún nivel de independencia de estos factores; es decir, buscaremos estimaciones generales ampliamente válidas.

3. Asintotas

Por una parte necesitamos analizar la potencia de los algoritmos independientemente de la potencia de la máquina que los ejecute e incluso de la habilidad del programador que los codifique. Por otra, este análisis nos interesa especialmente cuando el algoritmo se aplica a problemas grandes. Casi siempre los problemas pequeños se pueden resolver de cualquier forma, apareciendo las limitaciones al atacar problemas grandes. No debe olvidarse que cualquier técnica de ingeniería, si funciona, acaba aplicándose al problema más grande que sea posible: las tecnologías de éxito, antes o después, acaban llevándose al límite de sus posibilidades.

Las consideraciones anteriores nos llevan a estudiar el comportamiento de un algoritmo cuando se fuerza el tamaño del problema al que se aplica. Matemáticamente hablando, cuando N tiende a infinito. Es decir, su comportamiento asintótico.

Sean " $g(n)$ " diferentes funciones que determinan el uso de recursos. Habrá funciones " g " de todos los colores. Lo que vamos a intentar es identificar "familias" de funciones, usando como criterio de agrupación su comportamiento asintótico.

A un conjunto de funciones que comparten un mismo comportamiento asintótico le denominaremos un 'orden de complejidad'. Habitualmente estos conjuntos se denominan O , existiendo una infinidad de ellos.

Para cada uno de estos conjuntos se suele identificar un miembro $f(n)$ que se utiliza como representante de la clase, hablándose del conjunto de funciones " g " que son del orden de " $f(n)$ ", denotándose como

$$g \in O(f(n))$$

Con frecuencia nos encontraremos con que no es necesario conocer el comportamiento exacto, sino que basta conocer una cota superior, es decir, alguna función que se comporte "aún peor".

La definición matemática de estos conjuntos debe ser muy cuidadosa para involucrar ambos aspectos: identificación de una familia y posible utilización como cota superior de otras funciones menos malas:

Dícese que el conjunto $O(f(n))$ es el de las funciones de orden de $f(n)$, que se define como

$$O(f(n)) = \{g: \text{INTEGER} \rightarrow \text{REAL}^+ \text{ tales que} \\ \text{existen las constantes } k \text{ y } N_0 \text{ tales que} \\ \text{para todo } N > N_0, \quad g(N) \leq k \cdot f(N) \}$$

en palabras, $O(f(n))$ está formado por aquellas funciones $g(n)$ que crecen a un ritmo menor o igual que el de $f(n)$.

De las funciones " g " que forman este conjunto $O(f(n))$ se dice que "**están dominadas asintóticamente**" por " f ", en el sentido de que para N suficientemente grande, y salvo una constante multiplicativa " k ", $f(n)$ es una cota superior de $g(n)$.

3.1. Órdenes de Complejidad

Se dice que $O(f(n))$ define un "**orden de complejidad**". Escogeremos como representante de este orden a la función $f(n)$ más sencilla del mismo. Así tendremos

$O(1)$ orden constante

$O(\log n)$ orden logarítmico

$O(n)$ orden lineal

$O(n \log n)$

$O(n^2)$ orden cuadrático

$O(n^a)$	orden polinomial ($a > 2$)
$O(a^n)$	orden exponencial ($a > 2$)
$O(n!)$	orden factorial

Es más, se puede identificar una jerarquía de órdenes de complejidad que coincide con el orden de la tabla anterior; jerarquía en el sentido de que cada orden de complejidad superior tiene a los inferiores como subconjuntos. Si un algoritmo A se puede demostrar de un cierto orden O_1 , es cierto que también pertenece a todos los órdenes superiores (la relación de orden 'cota superior de' es transitiva); pero en la práctica lo útil es encontrar la "menor cota superior", es decir el menor orden de complejidad que lo cubra.

3.1.1. Impacto Práctico

Para captar la importancia relativa de los órdenes de complejidad conviene efectuar algunas cuentas.

Sea un problema que sabemos resolver con algoritmos de diferentes complejidades. Para compararlos entre sí, supongamos que todos ellos requieren 1 hora de ordenador para resolver un problema de tamaño $N=100$.

¿Qué ocurre si disponemos del doble de tiempo? Notese que esto es lo mismo que disponer del mismo tiempo en un ordenador el doble de potente, y que el ritmo actual de progreso del hardware es exactamente ese:

"duplicación anual del número de instrucciones por segundo".

¿Qué ocurre si queremos resolver un problema de tamaño $2n$?

$O(f(n))$	$N=100$	$t=2h$	$N=200$
$\log n$	1 h	10000	1.15 h
n	1 h	200	2 h
$n \log n$	1 h	199	2.30 h
n^2	1 h	141	4 h
n^3	1 h	126	8 h
2^n	1 h	101	10^{30} h

Los algoritmos de complejidad $O(n)$ y $O(n \log n)$ son los que muestran un comportamiento más "natural": prácticamente a doble de tiempo, doble de datos procesables.

Los algoritmos de complejidad logarítmica son un descubrimiento fenomenal, pues en el doble de tiempo permiten atacar problemas notablemente mayores, y para resolver un problema el doble de grande sólo hace falta un poco más de tiempo (ni mucho menos el doble).

Los algoritmos de tipo polinómico no son una maravilla, y se enfrentan con dificultad a problemas de tamaño creciente. La práctica viene a decirnos que son el límite de lo "tratable".

Sobre la tratabilidad de los algoritmos de complejidad polinómica habría mucho que hablar, y a veces semejante calificativo es puro eufemismo. Mientras complejidades del orden $O(n^2)$ y $O(n^3)$ suelen ser efectivamente abordables, prácticamente nadie acepta algoritmos de orden $O(n^{100})$, por muy polinómicos que sean. La frontera es imprecisa.

Cualquier algoritmo por encima de una complejidad polinómica se dice "intratable" y sólo será aplicable a problemas ridículamente pequeños.

A la vista de lo anterior se comprende que los programadores busquen algoritmos de complejidad lineal. Es un golpe de suerte encontrar algo de complejidad logarítmica. Si se encuentran soluciones polinomiales, se puede vivir con ellas; pero ante soluciones de complejidad exponencial, más vale seguir buscando.

No obstante lo anterior ...

- ... si un programa se va a ejecutar muy pocas veces, los costes de codificación y depuración son los que más importan, relegando la complejidad a un papel secundario.
- ... si a un programa se le prevé larga vida, hay que pensar que le tocará mantenerlo a otra persona y, por tanto, conviene tener en cuenta su legibilidad, incluso a costa de la complejidad de los algoritmos empleados.
- ... si podemos garantizar que un programa sólo va a trabajar sobre datos pequeños (valores bajos de N), el orden de complejidad del algoritmo que usemos suele ser irrelevante, pudiendo llegar a ser incluso contraproducente.

Por ejemplo, si disponemos de dos algoritmos para el mismo problema, con tiempos de ejecución respectivos:

algoritmo	tiempo	complejidad
f	100 n	$O(n)$
g	n^2	$O(n^2)$

asintóticamente, "f" es mejor algoritmo que "g"; pero esto es cierto a partir de $N > 100$. Si nuestro problema no va a tratar jamás problemas de tamaño mayor que 100, es mejor solución usar el algoritmo "g".

El ejemplo anterior muestra que las constantes que aparecen en las fórmulas para $T(n)$, y que desaparecen al calcular las funciones de complejidad, pueden ser decisivas desde el punto de vista de ingeniería. Pueden darse incluso ejemplos más dramáticos:

algoritmo	tiempo	complejidad
f	n	$O(n)$
g	100 n	$O(n)$

aún siendo dos algoritmos con idéntico comportamiento asintótico, es obvio que el algoritmo "f" es siempre 100 veces más rápido que el "g" y candidato primero a ser utilizado.

- ... usualmente un programa de baja complejidad en cuanto a tiempo de ejecución, suele conllevar un alto consumo de memoria; y viceversa. A veces hay que sopesar ambos factores, quedándonos en algún punto de compromiso.
- ... en problemas de cálculo numérico hay que tener en cuenta más factores que su complejidad pura y dura, o incluso que su tiempo de ejecución: queda por considerar la precisión del cálculo, el máximo error introducido en cálculos intermedios, la estabilidad del algoritmo, etc. etc.

4. Reglas Prácticas

Aunque no existe una receta que siempre funcione para calcular la complejidad de un algoritmo, si es posible tratar sistemáticamente una gran cantidad de ellos, basándonos en que suelen estar bien estructurados y siguen pautas uniformes.

Los algoritmos bien estructurados combinan las sentencias de alguna de las formas siguientes

1. sentencias sencillas
2. secuencia (;)
3. decisión (if)
4. bucles
5. llamadas a procedimientos

4.0. Sentencias sencillas

Nos referimos a las sentencias de asignación, entrada/salida, etc. siempre y cuando no trabajen sobre variables estructuradas cuyo tamaño este relacionado con el tamaño N del problema. La inmensa mayoría de las sentencias de un algoritmo requieren un tiempo constante de ejecución, siendo su complejidad $O(1)$.

4.1. Secuencia (;)

La complejidad de una serie de elementos de un programa es del orden de la suma de las complejidades individuales, aplicándose las operaciones arriba expuestas.

4.2. Decisión (if)

La condición suele ser de $O(1)$, complejidad a sumar con la peor posible, bien en la rama THEN, o bien en la rama ELSE. En decisiones múltiples (ELSE IF, SWITCH CASE), se tomara la peor de las ramas.

4.3. Bucles

En los bucles con contador explícito, podemos distinguir dos casos, que el tamaño N forme parte de los límites o que no.

Si el bucle se realiza un número fijo de veces, independiente de N, entonces la repetición sólo introduce una constante multiplicativa que puede absorberse.

```
for (int i= 0; i < K; i++) { algo_de_O(1) } => K*O(1) = O(1)
```

Si el tamaño N aparece como límite de iteraciones ...

```
for (int i= 0; i < N; i++) { algo_de_O(1) } => N * O(1) = O(n)
```

```
for (int i= 0; i < N; i++) {  
    for (int j= 0; j < N; j++) {  
        algo_de_O(1)  
    }  
}
```

tendremos $N * N * O(1) = O(n^2)$

```
for (int i= 0; i < N; i++) {  
    for (int j= 0; j < i; j++) {  
        algo_de_O(1)  
    }  
}
```

el bucle exterior se realiza N veces, mientras que el interior se realiza 1, 2, 3, ... N veces respectivamente. En total,

$$1 + 2 + 3 + \dots + N = N * (1+N) / 2 \rightarrow O(n^2)$$

A veces aparecen bucles multiplicativos, donde la evolución de la variable de control no es lineal (como en los casos anteriores)

```
c= 1;  
while (c < N) {
```

```

    algo_de_O(1)
    c= 2*c;
}

```

El valor inicial de "c" es 1, siendo " 2^k " al cabo de "k" iteraciones.

El número de iteraciones es tal que

$$2^k \geq N \Rightarrow k = (\log_2 N)$$

y, por tanto, la complejidad del bucle es $O(\log n)$.

```

c= N;
while (c > 1) {
    algo_de_O(1)
    c= c / 2;
}

```

Un razonamiento análogo nos lleva a $\log_2(N)$ iteraciones y, por tanto, a un orden $O(\log n)$ de complejidad.

```

for (int i= 0; i < N; i++) {
    c= i;
    while (c > 0) {
        algo_de_O(1)
        c= c/2;
    }
}

```

tenemos un bucle interno de orden $O(\log n)$ que se ejecuta N veces, luego el conjunto es de orden $O(n \log n)$

4.4. Llamadas a procedimientos

La complejidad de llamar a un procedimiento viene dada por la complejidad del contenido del procedimiento en sí. El coste de llamar no es sino una constante que podemos obviar inmediatamente dentro de nuestros análisis asintóticos.

El cálculo de la complejidad asociada a un procedimiento puede complicarse notablemente si se trata de procedimientos recursivos. Es fácil que tengamos que aplicar técnicas propias de la matemática discreta, tema que queda fuera de los límites de esta nota técnica.

5. Problemas P, NP y NP-completos

Hasta aquí hemos venido hablando de algoritmos. Cuando nos enfrentamos a un problema concreto, habrá una serie de algoritmos aplicables. Se suele decir que el orden de complejidad de un problema es el del mejor algoritmo que se conozca para resolverlo. Así se clasifican los problemas, y los estudios sobre algoritmos se aplican a la realidad.

Estos estudios han llevado a la constatación de que existen problemas muy difíciles, problemas que desafían la utilización de los ordenadores para resolverlos. En lo que sigue esbozaremos las clases de problemas que hoy por hoy se escapan a un tratamiento informático.

Clase P.-

Los algoritmos de complejidad polinómica se dice que son tratables en el sentido de que suelen ser abordables en la práctica. Los problemas para los que se conocen algoritmos con esta complejidad se dice que forman la clase P. Aquellos problemas para los que la mejor solución que se conoce es de complejidad superior a la polinómica, se dice que son problemas intratables. Sería muy interesante encontrar alguna solución polinómica (o mejor) que permitiera abordarlos.

Clase NP.-

Algunos de estos problemas intratables pueden caracterizarse por el curioso hecho de que puede aplicarse un algoritmo polinómico para comprobar si una posible solución es válida o no. Esta característica lleva a un método de resolución no determinista

consistente en aplicar heurísticos para obtener soluciones hipotéticas que se van desestimando (o aceptando) a ritmo polinómico. Los problemas de esta clase se denominan NP (la N de no-deterministas y la P de polinómicos).

Clase NP-completos.-

Se conoce una amplia variedad de problemas de tipo NP, de los cuales destacan algunos de ellos de extrema complejidad. Gráficamente podemos decir que algunos problemas se hayan en la "frontera externa" de la clase NP. Son problemas NP, y son los peores problemas posibles de clase NP. Estos problemas se caracterizan por ser todos "iguales" en el sentido de que si se descubriera una solución P para alguno de ellos, esta solución sería fácilmente aplicable a todos ellos. Actualmente hay un premio de prestigio equivalente al Nobel reservado para el que descubra semejante solución ... ¡y se duda seriamente de que alguien lo consiga!

Es más, si se descubriera una solución para los problemas NP-completos, esta sería aplicable a todos los problemas NP y, por tanto, la clase NP desaparecería del mundo científico al carecerse de problemas de ese tipo. Realmente, tras años de búsqueda exhaustiva de dicha solución, es hecho ampliamente aceptado que no debe existir, aunque nadie ha demostrado, todavía, la imposibilidad de su existencia.

6. Conclusiones

Antes de realizar un programa conviene elegir un buen algoritmo, donde por bueno entendemos que utilice pocos recursos, siendo usualmente los más importantes el tiempo que lleve ejecutarse y la cantidad de espacio en memoria que requiera. Es engañoso pensar que todos los algoritmos son "más o menos iguales" y confiar en nuestra habilidad como programadores para convertir un mal algoritmo en un producto eficaz. Es asimismo engañoso confiar en la creciente potencia de las máquinas y el abaratamiento de las mismas como remedio de todos los problemas que puedan aparecer.

En el análisis de algoritmos se considera usualmente el caso peor, si bien a veces conviene analizar igualmente el caso mejor y hacer alguna estimación sobre un caso promedio. Para independizarse de factores coyunturales tales como el lenguaje de programación, la habilidad del codificador, la máquina soporte, etc. se suele trabajar con un cálculo asintótico que indica como se comporta el algoritmo para datos muy grandes y salvo algún coeficiente multiplicativo. Para problemas pequeños es cierto que casi todos los algoritmos son "más o menos iguales", primando otros aspectos como esfuerzo de codificación, legibilidad, etc. Los órdenes de complejidad sólo son importantes para grandes problemas.

7. Bibliografía

Es difícil encontrar libros que traten este tema a un nivel introductorio sin caer en amplios desarrollos matemáticos, aunque también es cierto que casi todos los libros que se precien dedican alguna breve sección al tema. Probablemente uno de los libros que sólo dedican un capítulo; pero es extremadamente claro es

L. Goldschlager and A. Lister.

Computer Science, A Modern Introduction

Series in Computer Science. Prentice-Hall Intl., London (UK), 1982.